Projekat I

AIS analitika nad HDFS + Spark

Mihajlo Madić 2119

Big Data Systems

2026

Dataset: Piraeus AIS

- Korišćen je javni dataset *The Piraeus AIS Dataset for Large-scale Maritime Data Analytics*, objavljen u okviru Zenodo repozitorijuma.
- Dataset sadrži AIS poruke kreirane tokom aktivnosti brodova u okviru luke Pirej u Grčkoj.
- AIS zapisi opisuju kretanje brodova kroz atribute kao što su vreme, identifikator broda, geografska pozicija, brzina, kurs i pravac.
- Originalni dataset je dovoljno veliki za Big Data scenario; za ovaj prototip korišćen je kontrolisani podskup radi bržeg razvoja i ponovljivog benchmark-a.
- Izvor: https://zenodo.org/records/5792100

Zadatak

Implementirati ceo sistem:

1. priprema podataka i upload na HDFS
2. Spark aplikacija sa CLI upitima (`count`, `show`, `stats`)
3. benchmark poređenje režima izvršavanja

Rezultati projekta:

- reproducibilna infrastruktura kroz Docker Compose
- merenja performansi
- izveštaji sa rezultatima

Obim i podaci

Korišćen podskup:

- raw ulaz: ~971.54 MiB (jul + decembar 2018 CSV)
- preprocesirano: ~191.60 MiB (Parquet)

Relevantne HDFS putanje:

- /bds/proj1/raw
- /bds/proj1/processed/clean
- /bds/proj1/processed/quarantine
- /bds/proj1/processed/quality_report

Arhitektura

3-slojni model:

1. HDFS sloj: skladište (`namenode`, `datanode`)
2. Spark sloj: obrada (`spark-master`, `spark-worker-*`)
3. App sloj: submission i CLI (`app`)

Tok podataka

Raw CSV $\rightarrow$ HDFS `/raw` $\rightarrow$ preprocess job $\rightarrow$ HDFS `/processed/*`

- analytics CLI koristi `count`, `show`, `stats`
- benchmark kampanje koriste isti HDFS izvor

Docker Compose

Servisi:

- `namenode`, `datanode`
- `spark-master`, `spark-worker-1`, `spark-worker-2`
- `app`

Princip:

- isti HDFS izvor za oba benchmark moda
- `app` container submituje iste komande za `local[*]` i `spark://...`

Upload na HDFS

Reproducibilni checkpoint:

`automation/phase1_hdfs_upload.sh`

Ključne komande:

```
docker compose exec namenode hdfs dfs -mkdir -p /bds/proj1/raw
docker compose exec namenode hdfs dfs -put -f \
  /input/.../jul2018.csv \
  /bds/proj1/raw/jul2018.csv
docker compose exec namenode hdfs dfs -put -f \
  /input/.../dec2018.csv \
  /bds/proj1/raw/dec2018.csv
docker compose exec namenode hdfs dfs -count -h /bds/proj1/raw
```

Preprocessing (Spark job)

Reproducibilni checkpoint:

`automation/phase2_preprocessing.sh`

Job:

```
docker compose exec app /spark/bin/spark-submit \
  --master spark://spark-master:7077 \
  /workspace/scripts/preprocess_ais_hdfs.py \
  --input 'hdfs://namenode:9000/bds/proj1/raw/*.csv' \
  --output-base hdfs://namenode:9000/bds/proj1/processed
```

Preprocessing logika

Ključne transformacije:

```
missing_required = timestamp/vessel_id/lon/lat missing
invalid_geo      = lon/lat out of range
invalid_speed    = speed < 0
invalid_heading  = heading not in [0, 360]
invalid_course   = course not in [0, 360]

clean       <- no hard_fail
quarantine  <- hard_fail rows

clean:       parquet partitioned by year, month
quarantine:  parquet partitioned by reason
quality_report: summary + reason_counts
```

Analytics CLI

Komande:

- `count`
- `show`
- `stats` (min/max/avg/stddev + count)

Primeri:

```
ais_analytics.py count --year 2018 --month 12

ais_analytics.py show --year 2018 --month 7 --min-speed 15 \
  --sort-by timestamp --sort-desc --limit 20

ais_analytics.py stats --year 2018 --group-by month --metrics speed
```

Benchmark metodologija

Pravila za fer poređenje:

- isti dataset: `/processed/clean`
- ista komanda
- režimi: `--master local[*]` i `--master spark://spark-master:7077`

Kampanja:

- 5 iteracija
- svaka iteracija: 1 run po modu + poseban report
- zatim finalni agregat (5x2)

Benchmark alati i artefakti

Runner:

```
python3 scripts/benchmark_ais_analytics.py \
  --benchmark-name <name> \
  --modes both \
  --repeats 1 \
  --query-override "count ... | show ... | stats ..."
```

Aggregator:

```
python3 scripts/aggregate_benchmark_runs.py \
  --runs-dir runs/campaign_<id>/runs \
  --output-root runs/campaign_<id>/reports
```

Full checkpoint

Finalni reproducibilni sistem:

`automation/phase5_full_benchmarks.sh`

Šta radi:

1. podiže storage + compute + app sloj
2. puni HDFS i izvršava preprocessing
3. pokreće 4 benchmark kampanje
4. za svaku kampanju pravi 5 iteracionih reporta + 1 finalni agregat

Struktura rezultata

```
runs/campaign_<id>/
  runs/
    standalone/<timestamp>/
      meta.json
      results.json
      summary.json
      run_report.md
    cluster/<timestamp>/
      ...
  reports/
    <timestamp>/aggregate.md
    <timestamp>/aggregate.csv
    <timestamp>/aggregate.json
```

`meta.json` čuva raw i processed size snapshot.

Rezultati: Stats (speed)

Kampanja: `campaign_20260226T101745Z_stats_speed`

```
stats --year 2018 --group-by month --metrics speed \
  --order-by month --limit 12
```

mode	n	mean s	stddev s	min s	max s
standalone	5	11.049	0.576	10.350	11.781
cluster	5	15.444	0.458	15.023	16.202

Raw: `971.54 MiB`, Processed: `191.60 MiB`

Rezultati: Stats (heading, course)

Kampanja: `campaign_20260226T102011Z_stats_heading_course`

```
stats --year 2018 --group-by month --metrics heading,course \
  --order-by month --limit 12
```

mode	n	mean s	stddev s	min s	max s
standalone	5	11.132	0.498	10.760	11.825
cluster	5	16.069	0.611	15.484	16.734

Raw: `971.54 MiB`, Processed: `191.60 MiB`

Rezultati: Count

Kampanja: `campaign_20260226T102241Z_count`

`count --year 2018 --month 12`

mode	n	mean s	stddev s	min s	max s
standalone	5	9.486	0.018	9.465	9.508
cluster	5	14.065	0.231	13.657	14.188

Rezultati: Show

Kampanja: `campaign_20260226T102453Z_show`

```
show --year 2018 --month 7 --min-speed 15 \
  --sort-by timestamp --sort-desc --limit 20
```

mode	n	mean s	stddev s	min s	max s
standalone	5	10.490	0.031	10.445	10.513
cluster	5	14.309	0.220	14.167	14.697

Poređenje cluster vs standalone

Cluster/standalone odnos (mean):

benchmark	ratio	delta (s)
stats speed	1.397x	+4.395
stats heading, course	1.443x	+4.937
count dec2018	1.483x	+4.579
show fast jul2018	1.364x	+3.819

Analiza rezultata

Zaključci za trenutni obim (~1GB raw, ~192MB parquet):

- `local[*]` je brži u svim testovima.
- Cluster overhead dominira nad dobitkom paralelizacije na ovom obimu podataka.
- Stabilnost merenja zavisi od upita: `count` i `show` su stabilniji, dok `stats` ima veću varijansu.

Implikacija:

- Za dokaz da cluster postaje bolji, potrebno je povećati ulaz i/ili težinu transformacija.

Ograničenja i sledeći koraci

Ograničenja trenutnog eksperimenta:

- jedan obim podataka (~1GB raw)
- jedan host i ograničeni worker resursi
- fokus na latency, ne na throughput pod konkurentnim opterećenjem

Sledeće:

1. benchmark na većim obimima podataka (npr. 5GB+ raw)
2. podešavanje Spark parametara
3. finalna vizuelna analiza pomoću `aggregate.csv`

Reproducibilnost

Reference komande:

```
# HDFS + preprocess baseline
bash automation/phase2_preprocessing.sh

# jedna benchmark kampanja
python3 scripts/benchmark_ais_analytics.py ...
python3 scripts/aggregate_benchmark_runs.py ...
```

Sve kampanje i izveštaji su timestamped i append-only.

Pitanja?

Kraj